

AI641- AI for Robotics

Projects

You have 3 project to chose from, each corresponding to a different aspect of robotics as taught in the course. The 3 projects are:

- Real-time semantic segmentation + Vision-based heading controller.
- Reinforcement Learning-based multi-agent pursuit-evasion.
- Vision based navigation in ROS BARN environment.

Each project is explained below:

1 Real-time semantic segmentation + Vision-based heading controller

Points of Contact: Ibrahim, Musab, Sheza, Shazreen

This project combines 2 key aspects of modern robotics: Deep Learning, and Controls. You are given a 1280 by 720 RGB video feed at 30 FPS of an Intel RealSense D455 RGB-D camera. You have to train a light weight segmentation model that segments the image into 5 classes:

- Road
- Sidewalk
- Speed Breaker
- Human
- Obstacle

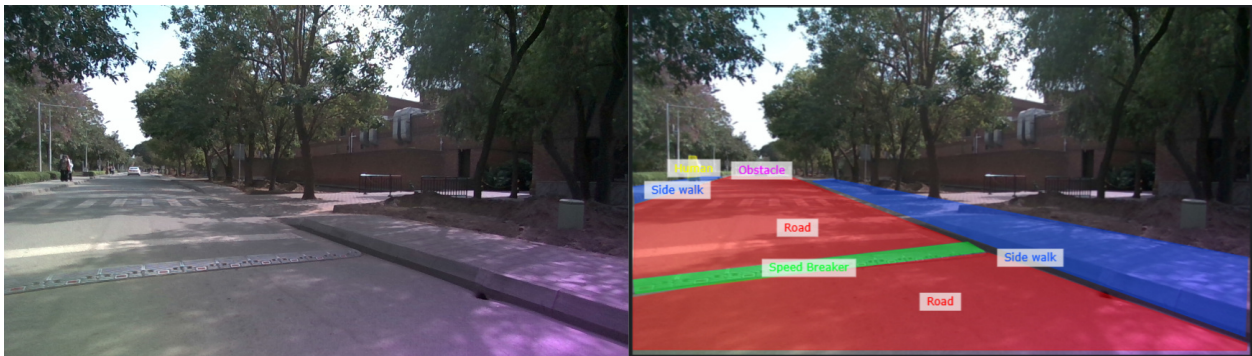


Figure 1: RGB and Segmentation Results

You have freedom in how you choose your segmentation model; it can be a model published in literature, or you can form one with custom architecture. Any approach you use, you will have to justify it in your report. In addition, since real-time data processing is of paramount importance in robotics, you must choose a model that can run real-time inference on the sample video.

Further, for real-time segmentation, you have to design a heading orientation (yaw) controller only using the RGB frames. For this purpose, calibration images are provided, from which calibration parameters can be estimated.

Finally, you will have to use the segmentation model to make decisions regarding:

- Decreasing speed when a speed breaker is near
- Stopping when an obstacle/human is near

Deliverables (In a Zip file):

- Report (including a link to the weights)
- Model code
- Code that takes any .mp4 video as input and uses the model and weights to generate a semantically segmented video file.
- A video showing your model running on the sample video in real time.
- A code that uses the segmentation model in combination with a heading controller to command appropriate actions such as:
 - Linear velocity increase/decrease.
 - Angular velocity increase/decrease.
 - Emergency stop.

Note: We will test your model on other similar videos as well, so make sure the models you submit are robust.

The data for this project can be found here

2 Reinforcement Learning-based multi-agent pursuit-evasion.

Points of Contact: Ibrahim, Fahd, Shazreen

You will be using the Unity Machine Learning Agents Toolkit (ML-Agents) if you choose this project, which is an open-source project that transforms the Unity game engine into a powerful laboratory for training intelligent AI. Traditionally, game AI is written using "if-then" logic (heuristics) or complex state machines. ML-Agents shifts this paradigm, allowing characters—or agents—to learn how to behave through trial and error using Reinforcement Learning (RL). By bridging the gap between Unity's high-fidelity physics simulations and Python's advanced machine learning libraries (like PyTorch), this toolkit allows students to develop autonomous NPCs, simulate robotics, and solve complex spatial puzzles without needing to manually program every possible movement.

Pre-requisites for this task:

- Python 3.10.12

Commands to set up unity ML-Agents

- Install unity hub from <https://docs.unity.com/en-us/hub>

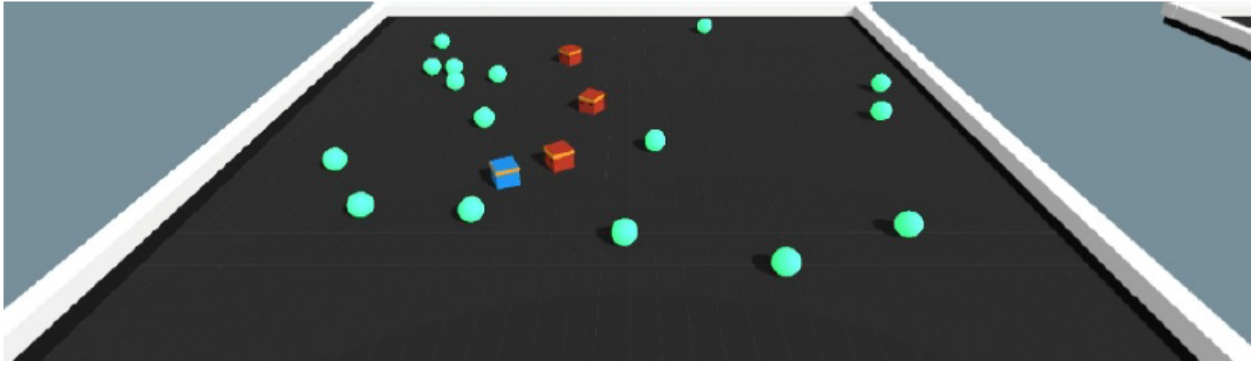


Figure 2: Unity Environment

- Install unity version 6000.3.9f1 from <https://docs.unity.com/en-us/hub/add-editor>
- Install Microsoft Visual C++ Redistributable: <https://learn.microsoft.com/en-us/cpp/windows/latest-supported-vc-redist?view=msvc-170>
- You can run multiple environments for unity-ML agents. For those wishing to train on their GPU, the following links are helpful:
 - <https://developer.nvidia.com/cudnn-downloads>
 - <https://developer.nvidia.com/cuda/toolkit>
- We recommend that you set up a dedicated python conda environment. Install miniconda from <https://www.anaconda.com/download>.
- Use these commands for setting up the environment:
 - `conda create -n mlagents python=3.10.12` (in anaconda powershell)
 - `conda activate mlagents` (activates the environment)
 - `conda install numpy=1.23.5`
 - `pip3 install torch =2.2.1 -index-url https://download.pytorch.org/whl/cu121`
- Verify installation by checking python:
 - `python`
 - `import torch`
 - `import numpy`
 - `print(torch.__version__)`
 - `print(numpy.__version__)`
 - `exit()`

Once the environment is setup, we need to setup the ML Agents.

- Download local copy of ML Agents from <https://github.com/Unity-Technologies/ml-agents> (Go to the releases page and download the source code zip file.)
- Copy the zip file into the Unity directory and extract. Rename the extracted folder to “ml-agents”
- go to ml-agents directory

- Install ML Agents from local file using these commands:
 - `python -m pip install ./ml-agents-envs`
 - `python -m pip install ./ml-agents`
- run `"mlagents-learn -help"` to check installation

If you get information on the possible inputs of the mlagents package, then you are good to go.

Afterwards, open the hub and select "Add Projects from Disk." Go into your ml-agents folder; inside you will find a folder named "Project." Select it, then click open. This will open the Unity editor. In the projects panels, click on the assets folder and then click on examples. Go into the basic folder and select the basic scene. A scene is essentially your environment. Keep the scene open and go back to the command prompt and run this command:

- `mlagents-learn config/ppo/Basic.yaml --run-id=run1 -`

Go back to the editor and press the play button at the top of the screen. Training will then begin.

Deliverables (in a zip file):

- Report
- Training set-up + custom environment

3 Vision based navigation in ROS BARN environment.

Points of Contact: Ibrahim, Musab, Sheza

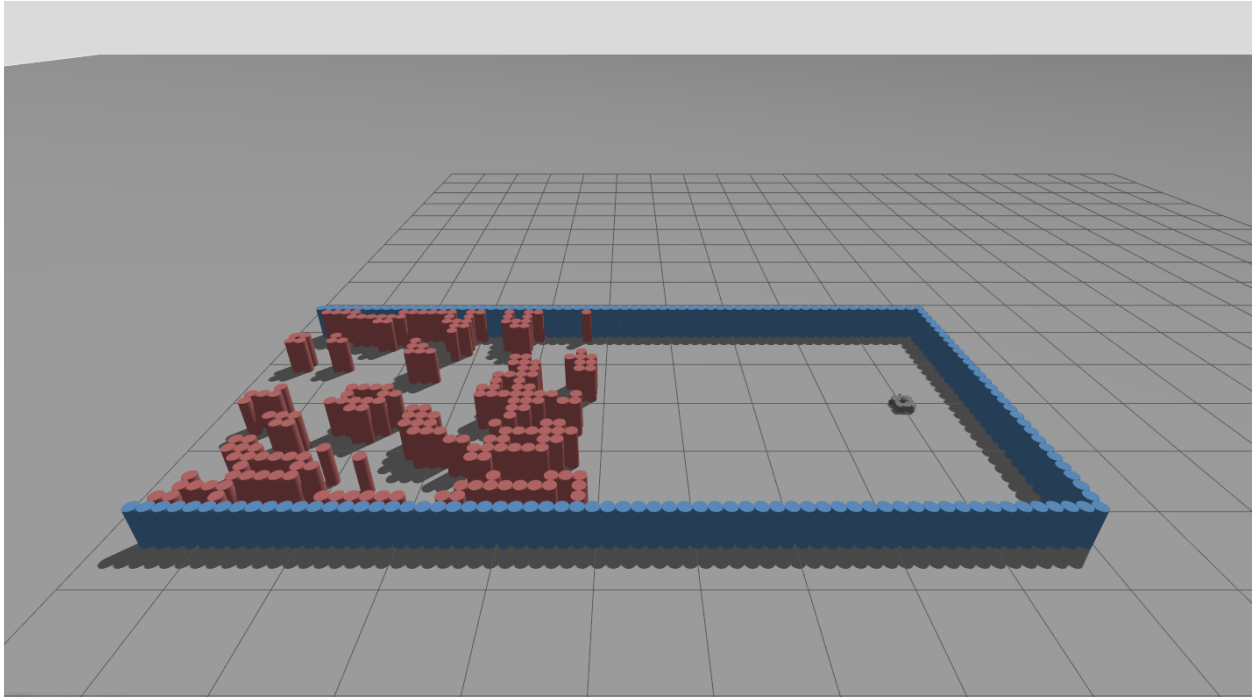


Figure 3: Project Environment

This project is inspired by the BARN challenge conducted at ICRA 2024, designed to evaluate path finding algorithms in highly constrained environments. The original challenge used a Clearpath

Jackal robot to navigate both simulated and physical world. We will be using turtlebot 3 waffle pi differential drive robot to navigate environments in simulation only.

To get started, you will need to make the environment for the challenge. You can download the worlds folder from here

The worlds were taken from this repo:

- <https://github.com/Saadmaghani/The-Barn-Challenge-Ros2>

Once you have your worlds, you can test them using gz sim :

- `gz sim /path/to/BARN/world`

To integrate the turtlebot, you will have to modify the `turtlebot3_world.launch` file, present in the location you installed the `turtlebot3_gazebo` package. In my case the path to the file was : `/opt/ros/jazzy/share/turtlebot3_gazebo/launch/turtlebot3_world.launch`.

In the file, comment out the `world` variable, and replace it with your path to a BARN world.

Note: You must use a world having index greater than 250 to allow for sufficient complexity.

You can use this command to launch the environment now:

- `ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py c_pose:=0 y_pose:=1`

Your gazebo world should appear as in 3:

You will now have to use the on-board sensors such as the camera and Li-Dar to orient the robot towards gaps and traverse between the red cylinders.

Deliverables (in a zip file):

- Report
- Project workspace.
- A video of robot traversal on 3 worlds having index greater than 250.

4 Project Timeline

- **Check point 1 (19 April)**

- **Real-time semantic segmentation + Vision-based heading controller.**

- * **Report Checkpoint:** Literature survey on segmentation models for path traversal, safe navigation, and vision based heading control. Problem setup
 - * **Demo checkpoint:** Segmentation model chosen and preliminary training done, work on heading control started.

- **Reinforcement Learning-based multi-agent pursuit-evasion.**

- * **Report Checkpoint:** Literature survey on multi-agent pursuit evasion. Problem setup.
 - * **Demo checkpoint:** Unity ML Agents environment setup. Enough training done should be done so we can see some crude form of intelligent decision making.

- **Vision based navigation in ROS BARN environment.**
 - * **Report Checkpoint:** Literature survey on vision based path navigation. Problem setup.
 - * **Demo checkpoint:** BARN environment setup, your robot should be able to traverse any world having index less than 10.
- **Check point 2 (2 May, Due date)**
 - **Real-time semantic segmentation + Vision-based heading controller.**
 - * **Report Checkpoint:** Checkpoint 1 report + results.
 - * **Demo checkpoint:** Segmentation model trained, heading controller working. Viva
 - **Reinforcement Learning-based multi-agent pursuit-evasion.**
 - * **Report Checkpoint:** Checkpoint 1 report + results.
 - * **Demo checkpoint:** Unity ML Agents fully trained. Agents should demonstrate intelligent decision making. Viva
 - **Vision based navigation in ROS BARN environment.**
 - * **Report Checkpoint:** Checkpoint 1 report + results.
 - * **Demo checkpoint:** Robot traversing any world having index greater than 250. Viva

For the report, use the IEEE double column conference template. The latex template can be found [here](#). No page limits.